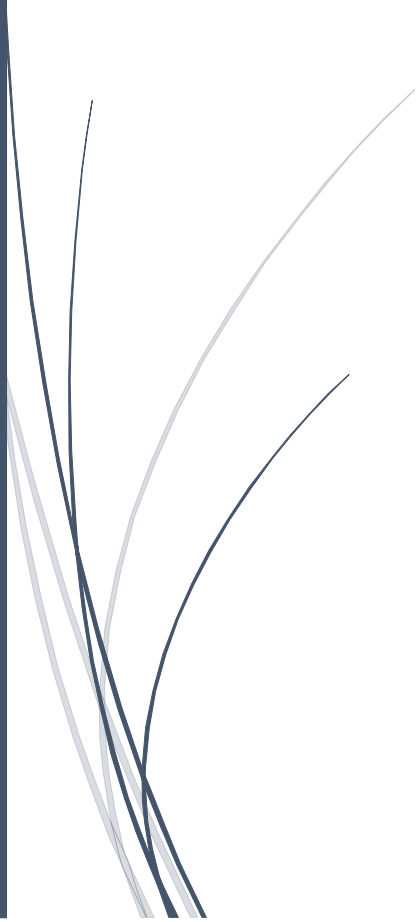
PROGRAMACIÓN
Tema 5

ESTRUCTURAS DE DATOS ESTÁTICAS
ARRAYS Y TABLAS



ÍNDICE

INTRODUCCIÓN....4

VARIABLES ESCALARES VS TABLAS....4

ÍNDICES....5

A. ÍNDICES FUERA DE RANGO....5

CONSTRUCCIÓN DE TABLAS....5

A. LONGITUD Y TIPO....5

B. VARIABLES DE TABLA....6

C. OPERADOR new....6

REFERENCIAS....7

A. RECOLECTOR DE BASURA....8

B. REFERENCIA NULL....8

USO DE TABLAS....8

TABLAS COMO PARÁMETROS DE FUNCIONES....10

OPERACIONES CON TABLAS: LA CLASE DE JAVA ARRAYS....10

A. OBTENCIÓN DEL NÚMERO DE ELEMENTOS DE UNA TABLA....10

B. INICIALIZACIÓN....10

C. RECORRIDO DE UN ARRAY DE TIPO VECTOR (TABLA DE UNA DIMENSIÓN)....11

D. MOSTRAR UNA TABLA....11

E. ORDENACIÓN....12

F. BÚSQUEDA....12

1. BÚSQUEDA EN UNA TABLA NO ORDENADA....12
2. BÚSQUEDA EN UNA TABLA ORDENADA....12

G. COPIA....12

H. INSERCIÓN....13

1. INSERCIÓN NO ORDENADA....13
2. INSERCIÓN ORDENADA....13

I. ELIMINACIÓN....14

1. TABLA NO ORDENADA....15
2. TABLA ORDENADA....15

J. COMPROBACIÓN (IGUALDAD) DE DOS TABLAS....16

TABLAS n-DIMENSIONALES....16

A. TABLAS BIDIMENSIONALES....16

B. TABLAS TRIDIMENSIONALES....17

C. TABLAS CON MÁS DIMENSIONES....17

CARACTERES Y CADENAS....18

CHARACTER....19

CLASE STRING.....19
BÚSQUEDA.....20
COMPROBACIONES.....21
CONVERSIONES Y SEPARACIÓN EN PARTES.....21

INTRODUCCIÓN

Una variable sólo puede almacenar un valor simultáneamente en cada instante de tiempo. Esto se denomina variable escalar.

Pero, ¿qué podemos hacer si necesitamos almacenar más de un valor de forma simultánea en una variable? En una primera aproximación alguien podría pensar que una solución es declarar una variable para cada valor, pero esta solución es tediosa y complicaría nuestro código exponencialmente. La respuesta adecuada es mediante el uso de tablas, las cuales reciben también el nombre de vectores o *arrays*.

Tenemos diferentes tipos de datos:

- Literales y constantes: Son los datos que se introducen entrecomillados en el caso de los textos o como expresiones númericas o booleanas que no van a cambiar a lo largo del proceso a ejecutar.
- Variables: Son datos que sí van a cambiar de valor en el tiempo. Estos a su vez se clasifican en:
    - Primitivos o simples
        - Short, byte, int y long.
        - Float, double
        - Char
        - Boolean
    - Compuestos
        - Estáticos
            - String
            - Vectores/Arrays/Tablas (se pueden denominar de las tres maneras)
        - Dinámicas (colecciones)
            - Listas
            - Pilas
            - Colas
            - Árboles
            - Mapas
            - Grafos

VARIABLES ESCALARES VS TABLAS

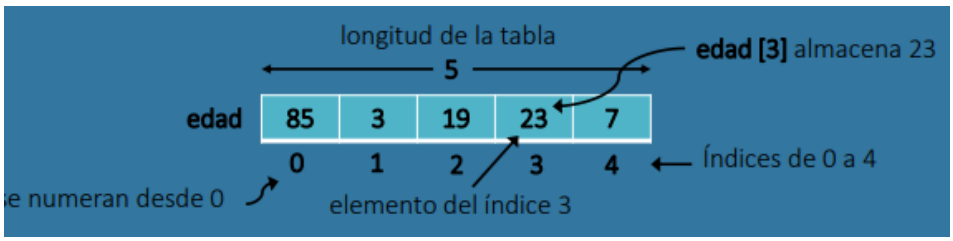
Podemos definir que una tabla es una variable que permite guardar más de un valor simultáneamente. Podríamos ver una tabla como una variable que es capaz de englobar a otras variables a las cuales llamaremos elementos o componentes referidas con un mismo nombre. Para que esto sea posible todas las variables deben ser del mismo tipo.

El uso de tablas está muy recomendado cuando debemos usar varios datos del mismo tipo de forma simultánea, por ejemplo, si tenemos que guardar 100 edades es necesario crear una tabla para almacenar todas las edades en lugar de crear 100 variables escalares (1 para cada valor, algo totalmente horrible de mantener, mejorar, cambiar).

Además, si de inicio no sabemos cuántos valores vamos a tener que guardar no nos quedará otra opción que usar tablas. Por ejemplo, un programa donde al inicio le preguntemos al usuario cuántas edades va a querer guardar, ese número N de edades no será un número fijo, sino que variará en cada uso del programa.

ÍNDICES

Es la forma de distinguir entre cada uno de los elementos o componentes que constituyen una tabla. Son una serie de números que se asignan en un orden para cada elemento comenzando por 0 y terminando por N-1 (si la tabla tiene 5 elementos, el último índice será el 4).



La forma de utilizar un elemento concreto de una tabla es por medio del nombre de la variable que identificará a la tabla junto al índice entre corchetes que distingue a ese elemento.

Por ejemplo, tenemos 4 edades en un array:

- Edad[0]. Sería el primer valor.
- Edad[1]. Sería el segundo valor.
- Edad[2]. Sería el tercer valor.
- Edad[3]. Sería el último valor.

A. ÍNDICES FUERA DE RANGO

Un índice fuera de rango es aquel que no existe, por ejemplo, antes hemos comentado un array con 4 edades yendo estas del 0 al 3, si pedimos la Edad[4] el programa será incapaz de encontrarla porque está fuera de rango, lo mismo si pedimos la Edad[-1]. En estos casos se obtendrá un error de tiempo de ejecución que provoca que el programa termine de forma inesperada (Runtime exception)

CONSTRUCCIÓN DE TABLAS

Las tablas cuando se declaran no tienen asignado el tamaño. Por lo tanto en el momento que determinamos que necesitaremos una tabla tenemos que:

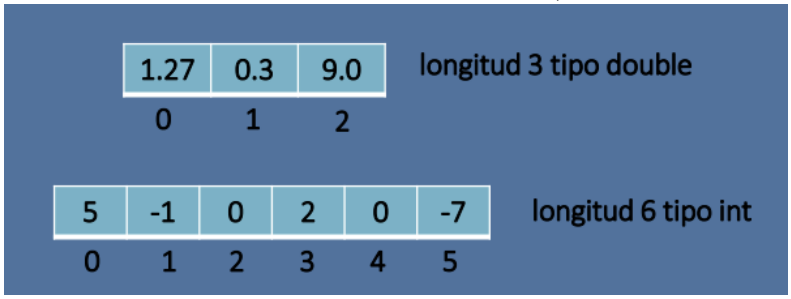
- Decidir qué tipo de datos vamos a almacenar y cuántos elementos necesitamos.
- Declarar una variable para la tabla.
- Crear la propia tabla.

A. LONGITUD Y TIPO

Una tabla se define por dos elementos:

- Longitud. Número de elementos que tiene.

- Tipo. Tipo de los datos que almacena (debe ser el mismo en todos los datos como se ha dicho más arriba).



B. VARIABLES DE TABLA

Lo primero que habrá que declarar es la variable utilizando corchetes, símbolo que diferencia entre una variable escalar y una tabla. Tenemos dos sintaxis:

```
tipo nombreVariable[ ];

tipo [ ] nombreVariable;
```

Por ejemplo, int edades[] y int [] edades son formas equivalentes para definir un vector. En este punto habremos declarado la tabla, pero no la hemos creado todavía.

C. OPERADOR new

Nos servirá para crear la tabla con la longitud adecuada. La sintaxis es la siguiente:

```
nombreVariable = new tipo[longitud];
```

Por ejemplo, edades = new int[5].

Por tanto, este operador va a construir la tabla donde todos los elementos estarán inicializados en 0 si es un array de tipo numérico o en false si es un array de tipo booleano.

Cualquier otro tipo se inicializará en null (null es un valor especial que significa "sin objeto asignado", lo veremos con profundidad cuando abordemos la POO)

También debemos tener en cuenta que se puede declarar y crear la tabla en una única sentencia:

```
int edad[ ] = new int[5];
```

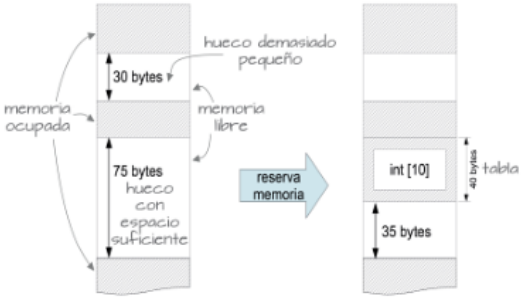
Existe una alternativa sintáctica que sólo será posible si conocemos el valor de todos los elementos que van a formar la tabla. En este caso

declararemos la tabla y luego entre llaves introduciremos los diferentes valores que va a tener la tabla.

```
int datos[ ] = {2, -3, 0, 7};
```

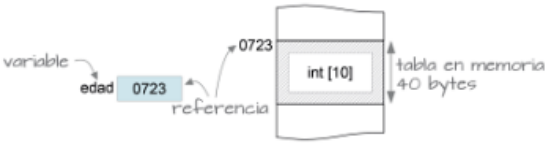
REFERENCIAS

Cada dato de un array se va a almacenar en memoria. Una vez que hayamos declarado y creado la tabla, se reservará el espacio en memoria necesario para completarlo, para ello multiplicará la cantidad de datos a guardar por el tipo de dato que se utiliza, por ejemplo, un int ocupa 4 bytes de memoria por lo que un array de 10 datos de tipo int ocupará 40 bytes de memoria.



Una vez que se vayan introduciendo los valores, esto se guardarán en cada dirección de memoria asignada. Cada una de esas direcciones de memoria es lo que se denomina referencia.

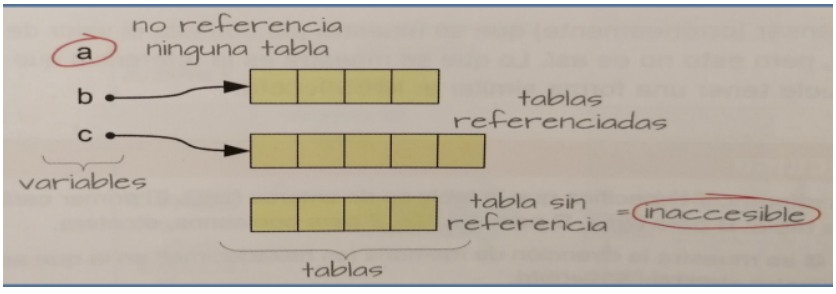
Por tanto, lo que se va a guardar realmente dentro del array no es el valor que se introduzca por consola sino la referencia (la dirección de memoria donde se está guardando la variable). Por eso también se les conoce como variables de referencia.



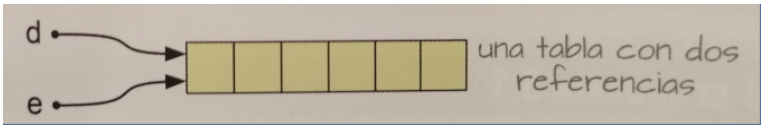
Es decir, si hacemos un array de "edades" con 4 índices con los valores 0, 1, 2 y 3 y solicitamos una impresión por consola de edades[1], en consola se verá el número de la segundad edad almacenada, que es el dato que se ha guardado en memoria, pero en realidad estamos accediendo a la referencia de la memoria RAM. Este tipo de accesos usando las referencias y su desplazamiento relativo con respecto a donde está almacenado el array son totalmente transparentes en Java, de forma que el programador se abstrae de este mecanismo (al contrario de lo que pasa en otros lenguajes como C/C++).

Estas referencias se modifican en cada ejecución, dependiendo de la ocupación de la memoria.

Si tenemos 3 variables array declaradas (por ejemplo, int a[], b[], c[]) e inicializamos b (b = new int [4]) y c (c = new int [5]) y new int [4]. La variable int a[] está declarada pero no hay ninguna referencia a ese array por lo que no hay posibilidad de almacenar datos, sin embargo b y c sí que son útiles.



Otro escenario posible es aquel en el que a través de dos variables diferentes accedamos a la misma tabla, por ejemplo declaramos dos variables int d[] e int e[] e inicializamos ambas de la siguiente manera: d = new int [6] y e = d. De esta manera las referencias de int d[] se guardarán también en int e[]. Esto no quiere decir que se copien los datos del array sino que se accede a las mismas posiciones de memoria desde dos variables diferentes.



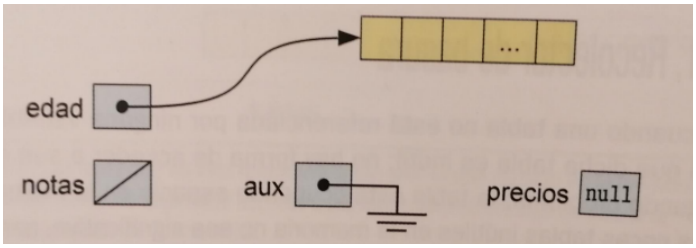
Esto sólo es posible hacerlo si los tipos de e y d son iguales, si fueran diferentes (por ejemplo, d un tipo int y e un tipo boolean) habría incompatibilidad de tipos y el programa provocaría un error.

A. <u>RECOLECTOR DE BASURA</u>

Java incorpora un sistema para impedir que las variables array declaradas, pero no iniciadas, sean borradas y no ocupen un espacio en memoria de forma inútil llegando a dejar grandes cantidades de memoria ocupadas e inaccesibles (bien por un mal diseño, bien por una mala intención).

Este sistema es el recolector de basura, Java de forma periódica comprueba que todas las tablas declaradas están construidas, si encuentra alguna que está declarada pero no construida la destruye dejando libre el espacio de memoria que estaba ocupando hasta ese momento. Este mecanismo también lo usa Java para reciclar memoria de las variables locales, referencias a objetos, y cualquier otra fuente de uso de memoria RAM.

B. <u>REFERENCIA NULL</u>



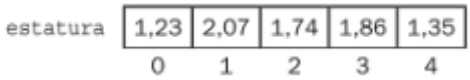
Cuando una referencia es asignada con el valor null, realmente estamos haciendo que la variable tenga una referencia vacía (nula) y por lo tanto si apuntaba a una zona de memoria que sólo tenía esa referencia, esta zona de memoria con todos sus datos, se reciclan, perdiendo la información que tuvieran.

USO DE TABLAS

Podemos trabajar con tablas de dos formas distintas:

- Sólo algunos elementos de la tabla almacenan información útil.
- Todos los elementos de la tabla almacenan información útil.

Hacer lo segundo va a simplificar la resolución de problemas en Java, por ejemplo, dada la siguiente tabla:

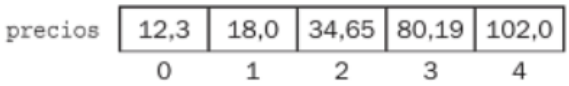


Todos los índices del array tienen un valor de referencia de tipo double que guarda una estatura.

En este caso una vez creadas las tablas, estas mantienen su longitud (de 0 a 4 en este caso) de forma constante y no es posible cambiarla. En el caso de que necesitemos aumentar el tamaño habrá que crear una segunda tabla que tenga una longitud mayor y, en ella, copiar los datos de la tabla original que nos interesen.

TABLAS ORDENADAS

Son tablas en las que los datos están ordenados en sentido creciente o decreciente, por ejemplo:



En estos casos deberemos tener cuidado con cómo insertar, cambiar o eliminar datos de la tabla para que esta continue de forma ordenada.

TABLAS + INDICADOR

Otra técnica para usar las tablas es suponer que no todos sus elementos almacenan datos. La tabla estará subdividida en dos partes: la primera con los elementos que almacenan datos y la segunda con los elementos vacíos.

Para ello, la tabla irá acompañada de una variable entera que funciona como indicador del número de datos que contiene la tabla, es decir, el indicador especifica cuántos elementos forman la primera parte (datos útiles) y el resto se consideran elementos vacíos.

Con esta técnica, cada vez que se inserta o elimina un dato en la tabla no es necesario modificar su longitud; en su lugar se modifica el indicador, lo que hace que el número de elementos con datos aumente o disminuya. Los elementos de una tabla siempre almacenan algún valor, por lo que los elementos que se consideran vacíos realmente contienen valores (denominados valores basura) que nunca tendremos en cuenta.

Aunque para esta unidad optaremos por usar tablas en las que todos los elementos contienen datos útiles, se ha comentado la técnica de tabla + indicador porque también la podéis encontrar si consultáis bibliografía. Su principal ventaja es que no necesita redimensionar continuamente la tabla, pero en cambio, debemos llevar un control manual del indicador.

TABLAS COMO PARÁMETROS DE FUNCIONES

Las tablas o arrays también se pueden pasar como parámetros de funciones.

Cuando usamos una variable a como parámetro b en la llamada a una función, el valor de la variable a se copia en el parámetro b (repasar el tema 4 si tenéis dudas). Si en el cuerpo de la función se modifica la variable b, se está modificando una copia, no la variable a, por lo que si tenemos que a contenía, por ejemplo, el valor 3, y dentro de la función tenemos la sentencia b++, podemos estar tranquilos porque la variable a es "externa" a la función y no cambiará, seguirá con el valor 3 aunque b contenga un 4.

Cuando utilizamos tablas como parámetros el mecanismo es parecido, pero no exactamente similar, y es importante entender la diferencia (pasará lo mismo cuando pasemos objetos como parámetros). El valor de la variable tabla utilizada en la llamada se copia al parámetro de la función (hasta aquí parece lo mismo). Pero en este caso, lo que se copia es la referencia de una tabla, con lo que se consigue que la tabla esté referenciada tanto por la variable utilizada en la llamada como por el parámetro. Esta es la gran diferencia, pues la modificación de un elemento de la tabla dentro de la función es visible desde la variable externa a la función, siendo persistente al salir de la función. Esto es normal, ya que disponemos de dos referencias, pero sólo tenemos una única tabla donde se realizan las modificaciones.

OPERACIONES CON TABLAS: LA CLASE DE JAVA ARRAYS

La API de Java contiene una clase que facilita el trabajo con arrays. Se trata de la clase Arrays que se integra dentro del paquete java.util.

A. OBTENCIÓN DEL NÚMERO DE ELEMENTOS DE UNA TABLA

Para conocer el número de elementos de un array utilizaremos el método length (realmente es una propiedad, no un método, pero podemos usarlo parecido a un método sin parámetros).

B. INICIALIZACIÓN

Utilizaremos la inicialización cuando queramos iniciar las variables del array con un valor diferente al que tienen por defecto.

Para ello utilizaremos el método fill, el cual es un método sobrecargado que nos permitirá varias opciones:

- Arrays.fill(nombreArray, valorDato). Por ejemplo, partiendo de un array de alturas de tipo double vamos a hacer que todas las edades estén inicializadas por defecto en 1.50m: Arrays.fill(arrayRlnAltura, 1.50).
- Arrays.fill(nombreArray, indiceInicio, indiceFin, valorDato). Utilizando el mismo dato vamos a hacer que los índices entre el 15 y el 20 se inicien en 1.50, el resto lo harán en 0.0: Arrays.fill(arrayRlnAltura, 15, 21, 1.50).

C. RECORRIDO DE UN ARRAY DE TIPO VECTOR (TABLA DE UNA DIMENSIÓN)

El patrón de código para recorrer una tabla de dimensión 1 (vector o array clásico es:

```
for (int i = desde; i <= hasta; i++) {

    // procesado de t[i]
    ...
}
```

La instrucción for tiene una sintaxis alternativa, conocida como for-each o for extendido, que permite recorrer los elementos de una tabla unidimensional. Con esta metodología usamos una variable objeto iterador, que el propio for se encarga de ir "renovando" en cada iteración del bucle:

```
for (declaración variable: tabla) {
    ...
}
```

D. MOSTRAR UNA TABLA

Mostrar una tabla consiste en mostrar sus elementos. Si ejecutamos:

```
int t[ ] = {8, 41, 37, 22, 19};
System.out.println(t); // muestra una referencia
}
```

No se muestra el contenido de la tabla; en su lugar se muestra la referencia que contiene t, que no es lo que queremos.

Para mostrar una tabla tendremos que realizar un recorrido en el podamos trabajar, uno a uno, cada elemento que la compone.

Por lo tanto tendremos que recorrer la tabla y mostrar sus elementos, de la siguiente forma:

```
for (int i = 0; i < t.length; i++) { //recorremos toda la tabla
   System.out.println(t[ i ]); // mostramos cada elemento
}
```

O utilizando for each:

```
for (int elemento: t) {
    System.out.println(elemento);
}
```

E. ORDENACIÓN

Ordenar una tabla consiste en cambiar de posición los datos que contiene para que, en conjunto, resulten ordenados.

Para ordenar una tabla haremos uso del método sort de la clase Arrays. Veremos el método sort en profundidad cuando estudiemos las colecciones en Java. Ahora mismo nos quedaremos con un ejemplo sencillo:

```
int edad = {85, 19, 3, 23, 7}; // tabla desordenada
Arrays.sort(edad); // ordena la tabla. Ahora edad = [3, 7, 19, 23, 85]
```

F. BÚSQUEDA

1. *BÚSQUEDA EN UNA TABLA NO ORDENADA*
   Se realizará una búsqueda secuencial recorriendo la tabla y comprobando los valores de los elementos (for con if interno). Esta búsqueda terminara cuando encontremos el valor o cuando hayamos recorrido todo el array. Veremos ejemplos distintos en los ejercicios.

2. *BÚSQUEDA EN UNA TABLA ORDENADA*
   Se utiliza una búsqueda dicotómica o búsqueda binaria utilizando el método de la clase Arrays binarySearch. Este método busca el valor que queremos dentro de un array ordenado y devuelve un valor entre 0 y el tamaño del array menos uno (si encuentra el elemento) que sería el índice del array en el que se encuentra ese valor o un valor negativo (si no encuentra el elemento) que informa de la posición donde tendría que colocarse el elemento buscado para que la tabla continúe ordenada.

G. COPIA

El procedimiento para realizar manualmente la copia exacta de una tabla consiste en

1. Crear una nueva tabla, que llamaremos destino o copia, del mismo tipo y longitud que la tabla original.
2. Recorrer la tabla original, copiando el valor de cada elemento en su lugar correspondiente en la tabla destino.

Sin embargo, Arrays proporciona esta funcionalidad mediante:

- Arrays.copyOf(nombreArray, nuevaLongitud). Construye un nuevo array de una nueva longitud.

```
int t[ ] = {1, 2, 1, 6, 23}; // tabla origen
int a[ ], b[ ]; // tabla destino
a = Arrays.copyOf(t, 3); // a = [1, 2, 1]
b = Arrays.copyOf(t, 10); // b = [1, 2, 1, 6, 23, 0, 0, 0, 0, 0]
```

- Arrays.copyOfRange(nombreArray, inicio, fin). Construye un nuevo array que copia los valores desde inicio (incluido) hasta fin (excluido).

```
int t[ ] = {7, 5, 3, 1, 0, -2};
int a[ ] = Arrays.copyOfRange(t, 1, 4); // a = [5, 3, 1]
```

H. INSERCIÓN

La forma de añadir un nuevo valor a una tabla depende de si está o no ordenada.

Si el orden no importa, basta con incrementar la longitud de la tabla e insertar el nuevo dato en el último elemento.

Y cuando la tabla está ordenada, hemos de insertar el nuevo dato de forma que todos los valores sigan ordenados.

1. *INSERCIÓN NO ORDENADA*
   Veamos el algoritmo para insertar el valor nuevo, en un elemento que añadimos al final de la tabla t. Hay que destacar que la longitud de la tabla no se modifica lo que realmente ocurre es que se crea una segunda tabla (copia de la tabla original) en la que hemos aumentado la longitud en uno.

   La nueva tabla se referencia con la misma variable t, dando la sensación de que la hemos modificado. La tabla original, al quedar sin referencia, queda a merced del recolector de basura. Este proceso es costoso tanto a nivel de ejecución como de coste en memoria y recolección de basura RAM. Es por ello que aunque aprendamos a usarlo, todo el mundo use en programas reales y eficientes estructuras de datos colecciones de Java o derivados

   La forma de añadir el elemento "nuevo" se muestra en el siguiente código:

```
t = Arrays.copyOf(t, t.length + 1); la copia incrementa la longitud
t[t.length – 1] = nuevo;
```

2. *INSERCIÓN ORDENADA*
   La inserción ordenada consiste en añadir un nuevo elemento en la tabla en la posición adecuada para que la tabla continúe ordenada.

   Primeramente, buscaremos el lugar que le correspondería al nuevo valor en la tabla; a este índice le llamaremos indiceInsercion. A continuación, crearemos una nueva tabla, que llamaremos copia, con un elemento extra.

Ahora hemos de copiar los elementos de la tabla original a la tabla copia, teniendo la precaución de no utilizar el elemento situado en indiceInsercion, que es un hueco que está reservado para el nuevo valor.

Es decir, todos los elementos cuyos índices son anteriores a indiceInsercion se copian en la misma posición, y los posteriores, se copian desplazados un elemento hacia el final de la tabla. Con esto conseguimos que, tras insertar el nuevo valor en el elemento marcado por indiceInsercion, la tabla se mantenga ordenada.

Finalmente, la copia será referenciada por la misma variable que referenciaba la tabla original, dando la sensación de que la tabla ha crecido. Veamos un ejemplo:

```
int t[ ] = {1, 2, 3, 4, 6, 7, 8};
int nuevo = 5;
int indiceInsercion = Arrays.binarySearch(t, nuevo);
// si indiceInsercion >= 0, el nuevo elemento (que está repetido) se inserta en
// en el lugar en que ya estaba, desplazando al original. Si por el contrario:
if (indiceInsercion < 0) { // si no lo encuentra
    // calcula donde debería estar
    indiceInsercion = -indiceInsercion – 1;
}
```

```
int copia[ ] = new int [t.length + 1]; // nueva tabla con longitud + 1
// copiamos los elementos antes del "hueco"
System.arraycopy(t, 0, copia, 0, indiceInsercion);
// copiamos desplazados los elementos tras el "hueco"
System.arraycopy(t, indiceInsercion, copia, indiceInsercion + 1,
t.length - indiceInsercion);
copia[ indiceInsercion ] = nuevo;  // asignamos el nuevo elemento
t = copia;  // t referencia la nueva tabla
System.out.println(Arrays.toString(t));  // mostramos
```

I. ELIMINACIÓN

Esta operación consiste en borrar un elemento de la tabla, por lo que después de una eliminación la longitud de la tabla decrece.

Antes de eliminar un elemento de una tabla, siempre tendremos que buscarlo para conocer en qué índice se encuentra.

Una vez localizado, la operación dependerá del tipo de tabla con la que estemos trabajando.

1. *TABLA NO ORDENADA*
   Para eliminar un elemento en una tabla, después de buscarlo, lo sustituimos por el último de la tabla (que ahora estará repetido). A continuación, creamos una copia de la tabla con los mismos datos, pero disminuyendo su longitud, lo que provoca que perdamos el último elemento, que es el que estaba repetido.

   Veamos el algoritmo donde t[ ] es la tabla con los datos e indiceBorrado contendrá, si existe, el índice del elemento que deseamos eliminar, que se almacena en la variable aBorrar.

```
… // algoritmo de búsqueda, que devuelve el índice del elemento a borrar si
   // existe, o -1 si no existe
if (indiceBorrado != -1) {  // encontrado
    t [indiceBorrado] = t [t.length – 1]; // copia el último en indiceBorrado
    t = Arrays.copyOf (t, t.length – 1];  // disminuimos la longitud de t
    System.out.println(Arrays.toString(t));  // mostramos
} else {
    … // no podemos borrar nada, ya que no lo hemos encontrado
}
```

2. *TABLA ORDENADA*
   El primer paso es buscar el elemento que se va a borrar (variable aBorrar). Al estar la tabla ordenada podemos utilizar la búsqueda dicotómica. Este algoritmo busca el índice (variable indiceBorrrado) que utilizaremos para determinar el elemento que eliminar.

   En tablas ordenadas, al eliminar un elemento tenemos que seguir manteniendo los demás valores contiguos y en el mismo orden. Para ello, tenemos que desplazar los valores que siguen a indiceBorrado una posición hacia el principio. Con esta técnica sobrescribimos el valor que borrar y obtenemos un hueco libre al final de la tabla, que desaparecerá al disminuir su longitud.

```
int t[ ] = {12, 27, 33, 38, 72, 92};
int aBorrar = new Scanner(System.in).nextInt( );
// usamos el algoritmo de búsqueda dicotómica
int indiceBorrado = Arrays.binarySearch(t, aBorrar);
if (indiceBorrar >= 0) {
// desplazamos los elementos posteriores a indiceBorrado
System.arraycopy(t, indiceBorrado + 1, t, indiceBorrado, t.length – indiceBorrado – 1);
t = Arrays.copyOf (t, t.length – 1); // disminuimos la longitud
System.out.println(Arrays.toString(t)); // mostramos
} else {
   … // no podemos borrar nada, ya que no lo hemos encontrado
}
```

J. COMPROBACIÓN (IGUALDAD) DE DOS TABLAS

El contenido de dos tablas no se puede comparar mediante el operador ==, ya que este operador no compara los elementos de las tablas, sino sus referencias. Por ejemplo:

```
int t1[ ] = {7, 9, 20};
int t2[ ] = {7, 9, 20}; // t1 y t2 tienen los mismos elementos
System.out.println(t1 == t2); // sin embargo muestra false
```

Ya que aunque t1 y t2 tienen los mismos elementos, tienen realmente distintas referencias (están ubicadas en distintas zonas de memoria) que es lo que el operador == está comparando.

Dos tablas se consideran iguales si contienen los mismos elementos en el mismo orden. Para comparar dos tablas disponemos del método de Arrays equals:

static boolean equals(tipo a[ ], tipo b[ ]): compara las tablas a y b, elemento a elemento. En el caso de que sean iguales devuelve true, y en caso contrario, false.

```
System.out.println(Arrays.equals(t1, t2)); // muestra true
```

TABLAS n-DIMENSIONALES

A. TABLAS BIDIMENSIONALES

Podemos ampliar el concepto de tabla haciendo que los elementos se extiendan en dos dimensiones, utilizando los ejes X e Y. Así la tabla poseerá longitud y anchura, o lo que es lo mismo, filas y columnas. Para identificar cada elemento de una tabla unidimensional hemos utilizado un índice, así que para las tablas bidimensionales, compuestas por filas y columnas, se necesita un par de índices [x][y]. Una tabla bidimensional recibe el nombre de matriz, aunque a diferencia de las matemáticas, en Java se numeran comenzando por 0 (al igual que los arrays o vectores).

La declaración de una tabla bidimensional se hace de la forma:

```
tipo nombreTabla[ ] [ ];
```

A continuación, creamos la tabla, indicando la longitud de cada dimensión. Veamos cómo crear la tabla datos que se corresponde con una matriz de 25 elementos con 5 filas y 5 columnas:

```
int datos[ ] [ ];
datos = new int [5] [5];
```

Los algoritmos que utilizan matrices de dos dimensiones requieren dos bucles anidados para poder recorrer sus elementos uno a uno. Un bucle se encarga del índice para la dimensión X y el otro para el índice del eje Y. Veamos un ejemplo para introducir por teclado la matriz datos:

```
for (i = 0; i < 5; i++) {  // eje X
    for (j = 0; j < 5; j++) { // eje Y
        datos [ i ] [ j ] = sc.nextInt( ); // leemos el elemento [ i ] [ j ]
    }
}
```

Para mostrar una tabla bidimensional podemos usar dos bucles anidados como los anteriores o bien utilizar el método estático Arrays deepToString. Por ejemplo, para mostrar la tabla datos.

```
System.out.println(Arrays.deepToString(datos));
```

Esta forma de sacar por pantalla una matriz nos muestra todos los datos individuales de la matriz en notación algebraica usando corchetes y comas. Por ejemplo, la matriz

2 3 6

4 3 8

9 2 2

Saldría en pantalla después de usar deepToString:

[[2, 3, 6], [4, 3, 8], [9 2 2]]

B. TABLAS TRIDIMENSIONALES

Añadiendo una nueva dimensión podemos crear tablas con anchura, altura y profundidad, es decir, tablas tridimensionales. Estas utilizan tres índices [x][y][z] para identificar cada elemento que la componen. Para poder recorrerlas y obtener los valores o modificarlos individualmente tenemos por lo tanto que usar tres bucles anidados, de la misma manera que usabamos dos para tablas de dos dimensiones.

C. TABLAS CON MÁS DIMENSIONES

Dibujar o imaginar una tabla de más de tres dimensiones es algo complicado. Nosotros vivimos en un mundo 4 dimensional, con tres dimensiones para localizar cada objeto en el espacio y una cuarta dimensión, el tiempo, para ver, por ejemplo, la evolución de un objeto en movimiento en el espacio tridimensional.

Pero más allá de nuestro mundo, es complicado representar, o siquiera imaginar, una tabla multidimensional. Un truco sencillo consiste en descomponer la tabla en otras más simples.

Por ejemplo, una tabla de 5 dimensiones puede verse como una tabla tridimensional, donde en cada elemento de la tabla se almacena una tabla bidimensional. De los cinco índices necesarios para identificar los elementos de una tabla 5 dimensional, podemos utilizar los tres primeros en la tabla tridimensional y utilizar los otros dos índices para situarnos en la segunda tabla bidimensional.

Pero el hecho de que no podamos dibujarla ni imaginarla no significa que no se puedan manipular sus elementos.

Las tablas n dimensionales son útiles para manejar la información atendiendo a criterios de clasificación. No es necesario que la información tenga una representación gráfica. Veamos un ejemplo: supongamos una máquina que procesa naranjas y necesitamos, por motivos de calidad, clasificarlas y llevar la cuenta del número de frutas recogidas de cada tipo, atendiendo a los criterios diámetro, color, maduración, forma y peso.

Al utilizar cinco criterios, lo más apropiado para almacenar los datos es una matriz 5 dimensional, haciendo corresponder cada dimensión con un criterio de clasificación. Falta, para cada una de las dimensiones (criterios) formalizar una correspondencia entre los posibles valores reales de un criterio (color naranja, amarillo o verde; nivel de maduración madura o inmadura; etc.) con las longitudes de cada dimensión, que utilizaremos como índices.

Pero teniendo en cuenta que la mayoría de los problemas de la realidad no tienen ese nivel de complejidad, en los ejercicios nos centraremos en tablas bi y tridimensionales.

CARACTERES Y CADENAS

Hasta ahora con los tipos primitivos sólo disponíamos del tipo char para trabajar con caracteres, pero su uso es un poco limitado.

Las cadenas de caracteres las definimos como un número determinado de caracteres, que pueden contener caracteres especiales como saltos de línea para formar párrafos distintos. Estos caracteres están internamente almacenados en una array de char, pero la Clase String de Java nos "oculta" los detalles de la gestión y almacenamiento de este "array interno" haciendo la gestión de los textos mucho más fácil al programador.

Las cadenas literales (constantes) se escriben entre comillas dobles, por ejemplo "esto es una cadena", mientras que los caracteres se escriben entre comillas simples, por ejemplo 'c'.

Un carácter se puede escribir como un literal carácter chrCaracter = 'a', como su código en decimal chrCaracter = 97 o mediante su valor en hexadecimal chrCaracter = '\u0061' (4 dígitos precedidos de \u).

Existen también caracteres especiales que se pueden escribir directamente con una barra invertida (\).

Carácter	Nombre
\b	Borrado a la izquierda
\n	Nueva línea
\r	Retorno de carro
\t	Tabulador
\f	Nueva página
\'	Comilla simple
\"	Comilla doble
\\	Barra invertida

En los últimos estándares de Unicode, los caracteres ocupan entre 1 y 4 bytes. En Java el tipo char, aunque es UTF-16 que es un standard Unicode, solo admite dos bytes, teniendo que usar clases especiales para trabajar con Unicode de 4 bytes, por ejemplo. Pero con el tipo char UTF-16 están cubiertos la inmensa mayoría de las lenguas en todo el mundo, lo cual es lo que necesitamos en nuestro curso.

CHARACTER

Existe una clase envoltorio llamada Character que permite realizar una serie de funciones que pueden facilitar el trabajo.

- Character.isDigit('a') – Devuelve true si el carácter es un dígito.
- Character.isLetter('a') – Devuelve true si el carácter es una letra.
- Character.isLetterOrDigit('a') – Devuelve true el carácter es un alfanumérico.
- Character.isLowerCase('a') – Devuelve true el carácter está en minúscula.
- Character.isUpperCase('a') – Devuelve true el carácter está en mayúscula.
- Character.isWhitespace('a') – Devuelve true si el carácter es un espacio en blanco (barra espaciadora, retorno de carro, nueva línea, tabulador…).
- Character.toLowerCase('a') – Convierte el carácter a minúscula, si no lo es lo deja tal cual.
- Character.toUpperCase('a') – Convierte el carácter a mayúscula, si no lo es lo deja tal cual.

Se pueden realizar también operaciones matemáticas sobre un tipo carácter, para que seleccione un carácter anterior o posterior, por ejemplo,

- System.out.println('h' + 'A' – 'a') – Devuelve una H.
- System.out.println('e' – '2') – Devuelve una c.

CLASE STRING

Las cadenas son secuencias de caracteres que se manipulan mediante la clase String.

Los literales de tipo String vendrán delimitados por comillas dobles "Hola Mundo". La definición de una variable de tipo String es similar a la de los tipos básicos, String strCadena = "Hola Mundo".

Es posible inicializar la cadena como un objeto más String strCadena = new String("Hola Mundo");

Cuando aplicamos el método valueOf(), de la clase String, sobre cualquier variable de tipo primitivo obtendremos una cadena que representa dicho objeto. Con los objetos se puede usar el método toString().

Para comparar cadenas no se pueden utilizar los operadores ==, >, <… es necesario usar métodos:

- equals – compara si son iguales dos cadenas strCadenal.equals(strCadena2);
- equalsIgnoreCase – Igual que equals pero ignorando mayúsculas y minúsculas
- strCadenal.equa1sIgnoreCase(strCadena2);
- regionMatches – Indica si se encuentra una cadena en otra. Se pasan como parámetros (ignora mayúsculas, minúsculas, posición en la cadena que invoca, segunda cadena, posición en la segunda, numero de caracteres a comparar). El primer parámetro es opcional. String strSaludo = "Hola Mundo", strPrueba = "Donde está Mundo"; strSaludo.regionMatches(5,strPrueba,11,5); devolverá true.
- compareTo – compara alfabéticamente dos cadenas, devolviendo: 0 si son iguales, <0 si la cadena que hace la llamada es menor y >0 si la cadena que hace la llamada es mayor. strCadena1.compareTo(strCadena2);
- compareToIgonreCase – Igual que el anterior, pero sin tener en cuenta mayúsculas y minúsculas.
- concat – Permite concatenar cadenas, aunque es raro su uso porque el operador + hace la misma función. strCadena1.concat(strCadena2);
- charAt – Devuelve el carácter indicado de una cadena. El primer carácter de una cadena es el 0.
- substring – devuelve una subcadena de la cadena original. Se puede indicar desde donde queremos que arranque la subcadena y que llegue hasta el final strCadena1.substring(4); También es posible indicar el principio y el final de la subcadena strCadena1.substring(4, 3);
- strip, stripLeading, stripTrailing – se utiliza para eliminar los espacios en blanco del principio y final de una cadena en el primer caso, sólo del principio en el segundo o solo del final en el tercero.
- length – indica el número de caracteres de una cedena. strCadena.length(); Se suele utilizar en bucles con una función charAt.

BÚSQUEDA

Dentro de una cadena es posible buscar caracteres o cadenas, en un sentido u otro y a partir de una posición indicada. Estas funciones devuelven el índice donde se ha localizado o -1 si no se encuentra.

- indexOf (carácter | cadena [, posición_inicial]) – Busca la primera ocurrencia de la cadena o el carácter. Si se indica la posición inicial,

empezará desde esta. La posición inicial se suele utilizar si queremos buscar otro carácter tras el primero.
- lastIndexOf (carácter | cadena [, posición_inicial]) – Busca la última ocurrencia de la cadena o el carácter.

COMPROBACIONES

Dentro de una cadena es posible utilizar métodos que me devolverán un booleano (verdadero, falso) al hacer una comprobación (¿está vacía la cadena?, ¿contiene esta subcadena?, ¿empieza o termina por?...)

- isEmpty – Indica si la cadena está vacía. strNombreCadena.isEmpty(); Devuelve true si está vacía y false si está llena.
- contains – Busca si dentro de la cadena contiene la subcadena pasada, si es así devuelve true, sino devolverá false. strNombreCadena.contains(strNombreSucadena);
- startsWith – Devuelve true si la cadena comienza por la subcadena pasada. Si se tiene la posición de inicio, mirará desde esa posición. strNombreCadena.startsWith(strNombreSubcadena [, intPosicionInicio]);
- endsWith – Igual que el caso anterior pero ahora mira por donde termina la cadena. strNombreCadena.endsWith(strNombreSubcadena)

CONVERSIONES Y SEPARACIÓN EN PARTES

- replace – Devuelve la cadena pasada, donde se han sustituido todas las ocurrencias del carácter original por el sustituto. strNuevaCadena = strNombreCadena.replace(chrCaracterOrigen, chrCaracterSustituto); En lugar de caracteres se pueden introducir cadenas a sustituir strNuevaCadena = strNombreCadena.replace(strCadenaOrigen, strCadenaSustituta);
- split – Devuelve un array de String que contiene las cadenas entre separadores. strArrCadenas = strNombreCadena.split(chrCaracterSeparador); Si es un carácter especial usar \\ antes del carácter.
- toCharArray – Devuelve un array de caracteres con los caracteres de la cadena que hace la llamada. chrArrCaracteres = strNombreCadena.toCharArray();
- valueOf – Devuelve de un array de caracteres el String formado por todos los caracteres del array. strNuevaCadena = String.valueOf(chrArrCaracteres[, intInicio, intNumCaract]); Se puede indicar desde donde se empieza con intInicio y cuantos caracteres queremos coger con intNumCaract.